`fkcompute`, an efficient F_K invariant calculator

Paul Orland, Davide Passaro, Lara San Martín Suárez, Toby Saunders-A'Court, Josef Svoboda

February 2026

Contents

1 Introduction

Modern topology increasingly relies on the computation of invariants to distinguish, classify, and analyze geometric and algebraic structures. As the field has progressed, these invariants themselves have grown in sophistication, and their computation has correspondingly become more intricate and resource-intensive, particularly as higher accuracy and larger datasets are demanded. At the same time, the generation and aggregation of large datasets of invariants has taken on new importance, not only for traditional mathematical investigation but also as a foundation for emerging data-driven approaches, including data science and machine learning methodologies, which are beginning to play a substantive role in contemporary mathematical research. Compounding these technical challenges, the proliferation of distinct invariants and the specialized algorithms required to compute them have created significant obstacles for effective distribution, reproducibility, and collaborative development, particularly when implementations involve highly complex and domain-specific codebases.

In this note, we focus on the F_K invariant, a quantum invariant of knots and links that was first introduced by Gukov and Manolescu [**?**]. Historically, the invariant emerged from the study of the $\widehat{Z}$ invariant, a q-series quantum invariant of closed 3-manifolds [**?**, **?**]. From this perspective, the F_K invariant can be interpreted as a $\widehat{Z}$ invariant for knot and link complements. More broadly, the $\widehat{Z}$ invariant itself was introduced as part of a program aimed at developing categorified invariants of 3-manifolds, analogous in spirit to how Khovanov homology categorifies the Jones polynomial for knots, and F_K plays a central role in this framework.

For the reasons outlined above, we introduce `fkcompute`, a computational framework designed to calculate F_K invariants starting from a braid presentation of a knot or link. `fkcompute` provides a suite of utilities and algorithms that implement the necessary algebraic and combinatorial procedures to construct the invariant directly from braid data. The implementation supports homogeneous braids and, more generally, the class of *nice* knots and links as defined by Park [**?**], which conjecturally includes all fibered knots; thereby covering a broad and practically relevant class of examples. The software has been tested extensively on knots and links with up to 12 crossings, demonstrating both robustness and computational feasibility within this range. To ensure correctness, computed results are validated against the inverse Alexander polynomial, which is known to agree with the corresponding expansion of the F_K invariant near $q = 1$ [**?**]. In the case of knots, an additional consistency check computes the first order term in the Melvin–Morton–Rozansky expansion of the colored Jones function [**?**, **?**] [L: proper citations: BarNatan-

Gar, Melvin-Morton.], for which we use an efficient polynomial-time algorithm[1] from [?].

`fkcompute` is provided both as a Python package and as a standalone command-line interface, allowing users to compute the F_K invariant directly from a braid presentation in a flexible and accessible manner. The package is designed with extensibility as a central goal: its source code is publicly available and structured to facilitate modification, extension, and integration into future computational workflows or research projects. Users may interact with `fkcompute` programmatically by importing it into Python scripts, enabling automated computation, batch processing, and integration with existing mathematical software pipelines. Alternatively, `fkcompute` can be used as a command-line tool, providing a straightforward interface for direct computation from the terminal without requiring additional programming. For users working in Mathematica, we also provide a Wolfram Language Paclet (`FkCompute`) which calls the Python package, allowing computations and symbolic expressions to be used directly in notebooks.

`fkcompute` is freely available for download through its public GitHub repository, providing open access to both the source code and accompanying documentation. Precomputed F_K tables used for validation and benchmarking are distributed separately from the core source repository. Internally, `fkcompute` leverages high-performance external libraries to ensure computational efficiency and reliability: polynomial arithmetic is handled using the FLINT library, a C++ library optimized for fast and precise computation with polynomials and power series, while Gurobi is used for constraint validation, enabling robust and efficient resolution of the linear and combinatorial conditions that arise in the construction of the invariant.

[1] We note that the relation of the invariant defined there to the MMR expansion is still conjectural [L: (at the time we wrote this note?)].

2 F_K background

In this section, we will briefly review the basics of F_K computation with R matrices. This perspective was pioneered by Park [?, ?]. The R matrix computation for the F_K naturally arises from an understanding of the F_K as an "analytically continued" version of the Jones polynomial, in the sense of Conjecture 1.5 of [?], whereby

$$\frac{F_K(x;q=e^{\hbar})}{x^{\frac{1}{2}}-x^{-\frac{1}{2}}}=\sum_{j\geq 0}\frac{P_K(j;x)}{\Delta_K(x)^{2j+1}}\frac{\hbar^j}{j!} \tag{1}$$

where the right hand side is the MMR expansion, or the large color expansion, of reduced colored Jones polynomials $J_K(n;q=e^{\hbar})$.

1. $U_q(\mathfrak{sl}_2)$ and classical R matrix [L: keep to minimum]
   - Basics of $U_q(\mathfrak{sl}_2)$: group elements, commutator and coproduct
   - R matrix, braid group, Yang-Baxter
   - R matrix for $U_q(\mathfrak{sl}_2)$
   - Colored Jones polynomials as traces of the R matrix

Both the colored Jones polynomials and the F_K can be calculated from R matrices on $U_q(\mathfrak{sl}_2)$. $U_q(\mathfrak{sl}_2)$ is the associative algebra generated by e, f, $q^{\frac{1}{2}}$, with relations

$$\begin{gathered} q^{i\frac{h}{2}}q^{j\frac{h}{2}}=q^{(i+j)\frac{h}{2}},\ q^{0\frac{h}{2}}=1 \\ q^{\frac{h}{2}}eq^{\frac{h}{2}}=qe,\ q^{\frac{h}{2}}fq^{\frac{h}{2}}=q^{-1}f, \\ [e,f]=\frac{q^{\frac{h}{2}}-q^{-\frac{h}{2}}}{q^{\frac{1}{2}}-q^{\frac{1}{2}}} \end{gathered}. \tag{2}$$

- F_K from R matrices
  - Verma modules and R matrices on Verma modules
  - Link case
- State sum and convergence
  - Homogeneous braid knots - all ok
  - Sign diagrams for the inverted state sum
  - Theorem (MMR)

3 F_K computation

The implementation in `fkcompute` follows the R-matrix/state-sum approach of [**?**, **?**] and is organized as a three-phase pipeline. The Python layer orchestrates the discrete choices (sign/inversion data and constraint preparation), while a compiled C++ backend performs the expensive enumeration and polynomial arithmetic.

Phase 1 (inversion / sign diagrams). Given a braid word and a target truncation degree, the first step is to find a *sign diagram* (called a *sign assignment* in the code) for which the inverted state sum converges to the requested degree. For homogeneous braids this data is determined directly from the braid word, using the rossing relations of [**?**]. Otherwise, `fkcompute` searches over sign assignments, trying braid variants obtained by cyclic shifts and horizontal reflections. For each fixed variant, the corresponding sign-search space is exhausted before moving on to the next variant. Candidate assignments are enumerated by a bitstring encoding and tested in parallel using Python multiprocessing. Each candidate is checked in two stages: first a fast local consistency test determines the R-matrix type at every crossing and rejects incompatible sign patterns, and then a global feasibility test reduces the induced linear relations and uses an integer-feasibility check. This step leverages Gurobi to certify that all degree constraints are simultaneously satisfiable.

Phase 2 (constraint system and ILP export). After a valid sign assignment is found, `fkcompute` builds the constraint system that encodes the admissible states of the inverted sum. The relations combine the open strand condition, forcing the open strand to be 0 or -1 at the endpoints, periodicity relations coming from the braid closure, sign bounds on every state variable, and crossing relations determined by the R-matrix types together with conservation laws. A reduction pass propagates constants and aliases and removes redundant relations, yielding a smaller system with a controlled set of free integer parameters. The reduced inequalities and the explicit linear expressions of all state variables in terms of these parameters are then exported to a compact CSV file. This file is the sole input to the C++ computation backend.

Phase 3 (enumeration and R-matrix evaluation in C++). The executable `fk_main` reads the exported CSV, validates that the inequalities actually bound all free variables (and, if needed, searches for an equivalent bounded set of criteria by combining inequalities), and then enumerates all admissible integer points. For each point, the linear expressions are evaluated to obtain the indices (i, j, i', j') at every crossing, and the corresponding R-matrix contribution is assembled from explicit q-binomial and (inverse) q-Pochhammer factors prescribed by the crossing type. These crossing factors are multiplied, truncated to the requested degree, and accumulated into the multivariable polynomial in x (one variable per link component) and q. This stage is parallelized with OpenMP and uses FLINT-backed polynomial arithmetic, with caching of frequently reused crossing factors. A final normalization is applied (multiplication by $-1 + x_0$ and truncation), and the result is written to JSON for consumption by the Python layer with optional symbolic formatting.

Overall, this separation keeps the mathematically delicate search and constraint preparation in readable Python code, while delegating the large-scale enumeration and polynomial arithmetic to optimized, multi-threaded C++.

4 Benchmarks

5 F_K tables and where to find more